

Yesterday 3:37 PM

Multithreading Coding Questions

These are excellent 5-year interview questions:

Q1

Print:

1 2 3 4 5 ...

using two threads alternately.

Show more ▾



Absolutely. For a **5-year Java backend interview**, these are not just “write code” questions. You should be able to explain **why you chose** `synchronized`, `Lock`, `volatile`, `AtomicInteger`, `BlockingQueue`, `ExecutorService`, `CompletableFuture`, **etc.**

Below is an interview-ready version with **Java code + explanation + follow-up questions**.

Multithreading Coding Questions — 5 YOE

Q1. Print 1 2 3 4 5 ... using two threads alternately

Requirement

- Thread-1 prints odd numbers.
- Thread-2 prints even numbers.
- Output:

1 2 3 4 5 6 7 8 9 10



Interview solution using `wait()` / `notifyAll()`

</> Java



```
class AlternateNumberPrinter {

    private int number = 1;
    private final int limit = 10;

    public synchronized void printOdd() throws InterruptedException {

        while (number <= limit) {

            while (number % 2 == 0 && number <= limit) {
                wait();
            }

            if (number <= limit) {
                System.out.println(Thread.currentThread().getName()
                    + " -> " + number);

                number++;
                notifyAll();
            }
        }
    }

    public synchronized void printEven() throws InterruptedException {

        while (number <= limit) {

            while (number % 2 != 0 && number <= limit) {
                wait();
            }

            if (number <= limit) {
                System.out.println(Thread.currentThread().getName()
                    + " -> " + number);

                number++;
                notifyAll();
            }
        }
    }
}
```

Usage:

</> Java



```
public class Main {

    public static void main(String[] args) {

        AlternateNumberPrinter printer =
            new AlternateNumberPrinter();

        Thread t1 = new Thread(() -> {
            try {
                printer.printOdd();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Odd");

        Thread t2 = new Thread(() -> {
            try {
                printer.printEven();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Even");

        t1.start();
        t2.start();

    }
}
```

Key interview point

Use **while** around `wait()`, not **if**.

Why?

Because a thread can wake up even when its condition isn't satisfied. This is called **spurious wakeup**.

Q2. Producer-Consumer using BlockingQueue

This is one of the most important concurrency coding questions.

Solution

</> Java



```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class Producer implements Runnable {

    private final BlockingQueue<Integer> queue;

    Producer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {

        try {
            for (int i = 1; i <= 10; i++) {

                queue.put(i);

                System.out.println(
                    "Produced: " + i);

                Thread.sleep(100);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Consumer:

</> Java



```
class Consumer implements Runnable {

    private final BlockingQueue<Integer> queue;

    Consumer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {

        try {
            while (true) {
```

```

        Integer value = queue.take();

        System.out.println(
            "Consumed: " + value);
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}
}

```

Main:

</> **Java**



```

public class Main {

    public static void main(String[] args) {

        BlockingQueue<Integer> queue =
            new LinkedBlockingQueue<>(5);

        Thread producer =
            new Thread(new Producer(queue));

        Thread consumer =
            new Thread(new Consumer(queue));

        producer.start();
        consumer.start();
    }
}

```

Why BlockingQueue ?

It handles:

- blocking when queue is full
- blocking when queue is empty
- thread coordination
- thread safety

So you don't manually need:

</> **Java**



```
wait();  
notify();
```

Common follow-up

Q: What happens when queue is full?

`put()` blocks until space becomes available.

Q: What happens when queue is empty?

`take()` blocks until an element becomes available.

Q3. Create a thread-safe counter

Bad implementation

```
</> Java  
  
class Counter {  
  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
}
```

Why isn't this thread-safe?

Because:

```
</> Java  
  
count++;
```

is not one operation.

Conceptually:

```
read count  
add 1
```

```
write count
```

Two threads can interfere with each other.

Solution 1 — synchronized

</> Java



```
class Counter {  
  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public synchronized int getCount() {  
        return count;  
    }  
}
```

Solution 2 — AtomicInteger

Usually a better answer for a simple counter:

</> Java



```
import java.util.concurrent.atomic.AtomicInteger;  
  
class Counter {  
  
    private final AtomicInteger count =  
        new AtomicInteger(0);  
  
    public void increment() {  
        count.incrementAndGet();  
    }  
  
    public int getCount() {  
        return count.get();  
    }  
}
```

5-year interview answer

For a simple numeric counter, I would prefer `AtomicInteger` because the increment can be performed using CAS without locking. If the operation involves multiple shared variables or requires a compound invariant, I would use synchronization or a `Lock`.

Q4. Implement thread-safe Singleton

There are several approaches.

Best interview answer — Holder idiom

</> Java



```
class Singleton {  
  
    private Singleton() {  
    }  
  
    private static class Holder {  
        private static final Singleton INSTANCE =  
            new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return Holder.INSTANCE;  
    }  
}
```

Why is it thread-safe?

Class initialization is guaranteed by the JVM to be thread-safe.

`Holder` is initialized only when:

</> Java



```
getInstance()
```

accesses:

</> Java



`Holder.INSTANCE`

Double-checked locking

Another important interview solution:

</> Java



```
class Singleton {  
  
    private static volatile Singleton instance;  
  
    private Singleton() {  
    }  
  
    public static Singleton getInstance() {  
  
        if (instance == null) {  
  
            synchronized (Singleton.class) {  
  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
  
        return instance;  
    }  
}
```

Why volatile ?

This is a classic interview question.

Without `volatile`, another thread could potentially observe a reference before the object's initialization is safely published.

The important point:

`volatile` provides visibility and ordering guarantees; it does not make arbitrary compound operations atomic.

Q5. Implement a deadlock and fix it

Deadlock example

</> Java



```
class DeadlockExample {

    private final Object lock1 = new Object();
    private final Object lock2 = new Object();

    public void method1() {

        synchronized (lock1) {

            System.out.println("Thread 1 acquired lock1");

            synchronized (lock2) {
                System.out.println("Thread 1 acquired lock2");
            }
        }
    }

    public void method2() {

        synchronized (lock2) {

            System.out.println("Thread 2 acquired lock2");

            synchronized (lock1) {
                System.out.println("Thread 2 acquired lock1");
            }
        }
    }
}
```

Execution:

```
Thread 1:
lock1 → waiting for lock2

Thread 2:
lock2 → waiting for lock1
```



Both wait forever.

Fix — consistent lock ordering

Always acquire locks in the same order:

</> Java



```
public void method1() {  
  
    synchronized (lock1) {  
  
        synchronized (lock2) {  
  
            // business logic  
        }  
    }  
}  
  
public void method2() {  
  
    synchronized (lock1) {  
  
        synchronized (lock2) {  
  
            // business logic  
        }  
    }  
}
```

Now both threads follow:

lock1 → lock2



Therefore circular waiting is prevented.

Other ways to prevent deadlock

- consistent lock ordering
- reduce lock scope
- use `tryLock()` with timeout
- avoid nested locks
- use higher-level concurrency utilities

Q6. Execute 3 tasks concurrently and combine result

This is extremely common for backend interviews.

Suppose:

API 1 → User
API 2 → Orders
API 3 → Payments



They are independent, so execute them concurrently.

CompletableFuture solution

</> Java



```
import java.util.concurrent.CompletableFuture;

class Service {

    public String getUser() {
        return "User";
    }

    public String getOrders() {
        return "Orders";
    }

    public String getPayments() {
        return "Payments";
    }

    public void execute() {

        CompletableFuture<String> user =
            CompletableFuture.supplyAsync(this::getUser);

        CompletableFuture<String> orders =
            CompletableFuture.supplyAsync(this::getOrders);

        CompletableFuture<String> payments =
            CompletableFuture.supplyAsync(this::getPayments);

        CompletableFuture<String> combined =
            CompletableFuture.allOf(
                user,
                orders,
                payments
            ).thenApply(v ->
```

```
        user.join() + " | "  
        + orders.join() + " | "  
        + payments.join()  
    );  
  
    System.out.println(combined.join());  
}  
}
```

Output:

User | Orders | Payments



Important distinction

</> Java

`thenApply()`



transforms a result.

</> Java

`thenAccept()`



consumes a result.

</> Java

`thenRun()`



runs an action without receiving the result.

</> Java

`thenCompose()`



chains dependent asynchronous operations.

</> Java

`thenCombine()`



combines two independent futures.

Q7. Implement timeout for a task

Modern Java provides convenient timeout methods.

</> Java



```
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> {

        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        return "Success";
    });

String result = future
    .orTimeout(2, java.util.concurrent.TimeUnit.SECONDS)
    .exceptionally(ex -> "Timed out")
    .join();

System.out.println(result);
```

If the task doesn't complete within 2 seconds:

Timed out



orTimeout() VS completeOnTimeout()

orTimeout

Fails the future:

</> Java



```
future.orTimeout(2, TimeUnit.SECONDS);
```

completeOnTimeout

Returns a fallback value:

</> Java



```
future.completeOnTimeout(  
    "DEFAULT",  
    2,  
    TimeUnit.SECONDS  
);
```

This distinction is worth remembering for interviews.

Q8. Implement a custom rate limiter

Suppose an API allows:

5 requests per second



A simple token-bucket-style implementation can be built using synchronization.

</> Java



```
class RateLimiter {  
  
    private final int maxRequests;  
    private final long intervalMillis;  
  
    private int availableRequests;  
    private long lastRefillTime;  
  
    public RateLimiter(  
        int maxRequests,  
        long intervalMillis) {  
  
        this.maxRequests = maxRequests;  
        this.intervalMillis = intervalMillis;  
  
        this.availableRequests = maxRequests;  
        this.lastRefillTime = System.currentTimeMillis();  
    }  
  
    public synchronized boolean allowRequest() {  
  
        refill();  
  
        if (availableRequests > 0) {  
            availableRequests--;  
        }  
    }  
}
```

```

        return true;
    }

    return false;
}

private void refill() {

    long now = System.currentTimeMillis();

    if (now - lastRefillTime >= intervalMillis) {

        availableRequests = maxRequests;
        lastRefillTime = now;
    }
}
}

```

Usage:

 **Java**



```

RateLimiter limiter =
    new RateLimiter(5, 1000);

if (limiter.allowRequest()) {
    System.out.println("Request allowed");
} else {
    System.out.println("Request rejected");
}

```

Interview discussion

A production distributed rate limiter usually needs more than this:

```

Client
↓
Load Balancer
↓
Multiple application instances
↓
Distributed rate limiter
↓
Redis

```



Why Redis?

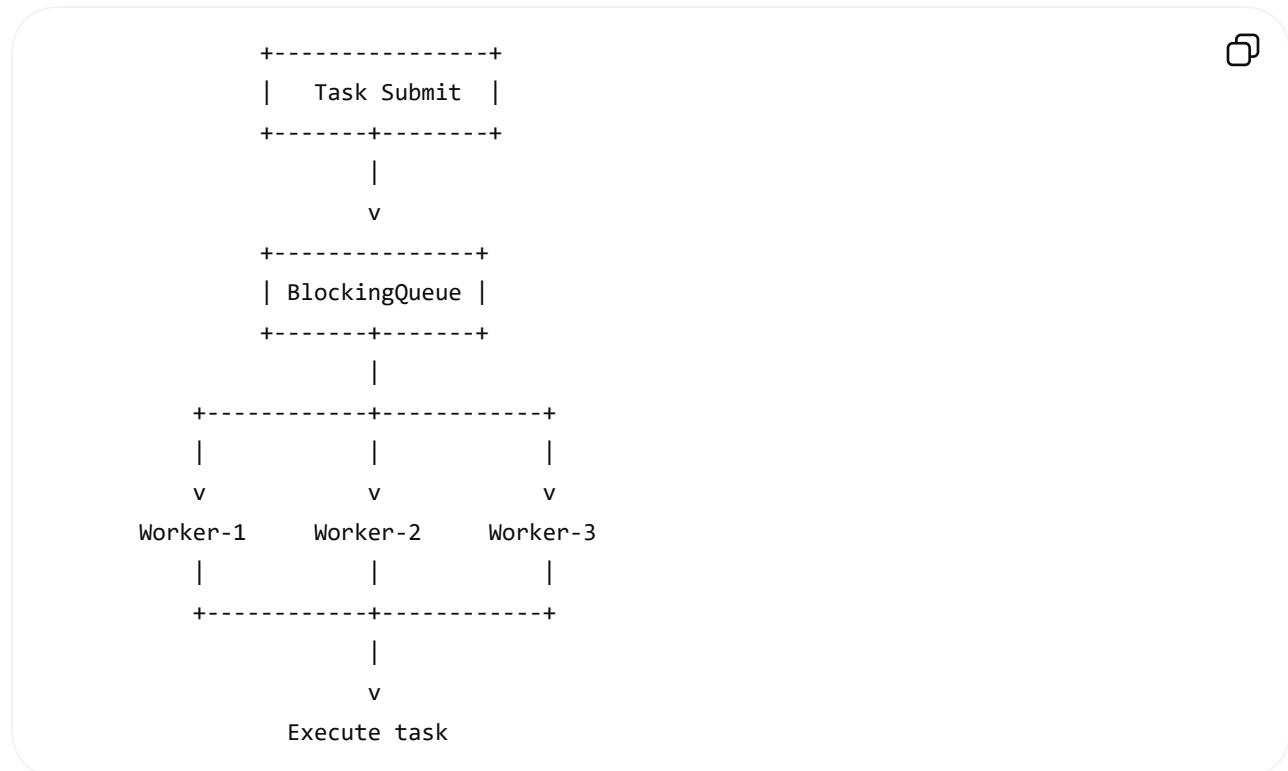
Because if each application instance maintains its own counter, the global limit isn't actually global.

Q9. Create a thread pool from scratch conceptually

This is a **very good 5-year question**.

Don't immediately start writing 200 lines of code.

First explain the architecture.



Core components

A basic thread pool requires:

1. Worker threads
2. Task queue
3. `execute()` method
4. Worker loop
5. Shutdown mechanism

Conceptual implementation

</> Java



```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class SimpleThreadPool {

    private final BlockingQueue<Runnable> queue;
    private final Worker[] workers;

    public SimpleThreadPool(int poolSize) {

        queue = new LinkedBlockingQueue<>();

        workers = new Worker[poolSize];

        for (int i = 0; i < poolSize; i++) {
            workers[i] = new Worker();
            workers[i].start();
        }
    }

    public void execute(Runnable task) {

        queue.offer(task);
    }

    private class Worker extends Thread {

        @Override
        public void run() {

            while (!Thread.currentThread().isInterrupted()) {

                try {
                    Runnable task = queue.take();
                    task.run();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }
    }
}
```

Usage:

</> Java



```
SimpleThreadPool pool =  
    new SimpleThreadPool(3);  
  
pool.execute(() ->  
    System.out.println("Task 1"));  
  
pool.execute(() ->  
    System.out.println("Task 2"));  
  
pool.execute(() ->  
    System.out.println("Task 3"));
```

Interview follow-ups

They may ask:

What happens when queue is full?

You need a rejection policy.

What happens during shutdown?

Stop accepting new tasks and decide whether queued tasks should finish.

How does `ExecutorService` improve this?

It provides production-ready lifecycle management, queueing, worker management, rejection policies, futures, etc.

Q10. Print ABCABCABCABC using 3 threads

This is one of the classic synchronization questions.

Thread 1:

A



Thread 2:

B



Thread 3:

c



Output:

ABCABCABCABC



Solution using `wait()` / `notifyAll()`

</> Java



```
class ABCPrinter {

    private int state = 0;
    private final int times = 4;

    public synchronized void printA() throws InterruptedException {

        for (int i = 0; i < times; i++) {

            while (state != 0) {
                wait();
            }

            System.out.print("A");

            state = 1;

            notifyAll();
        }
    }

    public synchronized void printB() throws InterruptedException {

        for (int i = 0; i < times; i++) {

            while (state != 1) {
                wait();
            }

            System.out.print("B");

            state = 2;

            notifyAll();
        }
    }
}
```

```

    }
}

public synchronized void printC() throws InterruptedException {

    for (int i = 0; i < times; i++) {

        while (state != 2) {
            wait();
        }

        System.out.print("C");

        state = 0;

        notifyAll();
    }
}
}

```

Usage:

</> Java



```

public class Main {

    public static void main(String[] args) {

        ABCPrinter printer = new ABCPrinter();

        Thread t1 = new Thread(() -> {
            try {
                printer.printA();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread t2 = new Thread(() -> {
            try {
                printer.printB();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread t3 = new Thread(() -> {
            try {

```

```
        printer.printC();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

t1.start();
t2.start();
t3.start();
}
}
```

Output:

ABCABCABCABC

📄

What the interviewer is actually testing

Question	Main concept
Q1	wait/notify , thread coordination
Q2	Producer-consumer, BlockingQueue
Q3	Atomicity, synchronized , CAS
Q4	Safe publication, volatile , class initialization
Q5	Deadlock and lock ordering
Q6	CompletableFuture , parallel execution
Q7	Timeout and async exception handling
Q8	Synchronization + distributed-system thinking
Q9	Thread pool internals
Q10	Multiple-thread coordination

The 5-year-level follow-up questions you should prepare

After writing each solution, expect the interviewer to move from **coding** → **internals** → **production design**.

For example, after Q3:

Interviewer: Why is `count++` not thread-safe?

Then:

What is CAS?

Then:

How does `AtomicInteger.incrementAndGet()` work?

Then:

When would you prefer `synchronized` over `AtomicInteger` ?

Then:

What happens under very high contention?

And finally:

How would you implement a distributed counter across multiple application instances?

That's the level I would prepare for a **5-year Java backend interview**.



ChatGPT can make mistakes. Check important info.